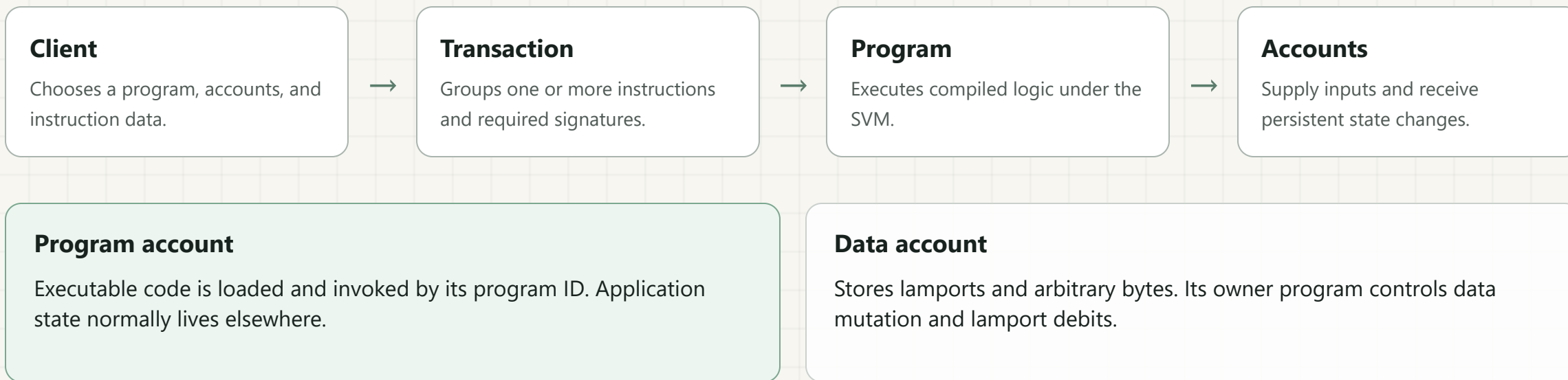


Solana SVM Blueprint

A compact reference for accounts, transactions, parallel execution, PDAs, CPI, compute limits, and the Anchor development loop.

Unofficial study material based on Solana School Fall 2026 Session 1 and verified against public Solana documentation. This artifact is not created, maintained, or endorsed by Solana Foundation.

Programs contain logic. Accounts contain state.



Solana programs are commonly described as stateless because persistent application data is passed through accounts rather than stored in program-local global state.

Persistent account fields and instruction privileges are different things

Persistent account fields

- `lamports` : the account balance
- `data` : arbitrary stored bytes
- `owner` : the program allowed to modify data or debit lamports
- `executable` : whether the account contains executable program code
- `rent_epoch` : retained as a deprecated field

Per-instruction metadata

- `is_signer` : a required signature was provided or signer privilege was granted by the runtime
- `is_writable` : the instruction may modify the account
- These flags are declared by the transaction and are not stored account fields.

The owner field is a program ID, not the person who holds a private key. A keypair controls signatures for its public address; it does not become an account's owner program.

Execution inputs are declared before the runtime starts

Instruction

- Target program ID
- Ordered account metadata
- Opaque instruction data

Transaction

- One or more instructions
- A recent blockhash
- Required signatures

Instruction A

Runs successfully



Instruction B

Returns an error



Transaction

All state changes are rolled back

Transactions are atomic for state changes. A failed transaction can still charge its transaction fee because validators performed verification and execution work.

Declared account access exposes conflicts before execution

Transaction	Declared access	Scheduling result
A	Reads account X	May run with other readers of X
B	Reads account Y, writes account Z	May run with A when their account sets do not conflict
C	Writes account X	Conflicts with A until exclusive access to X is available

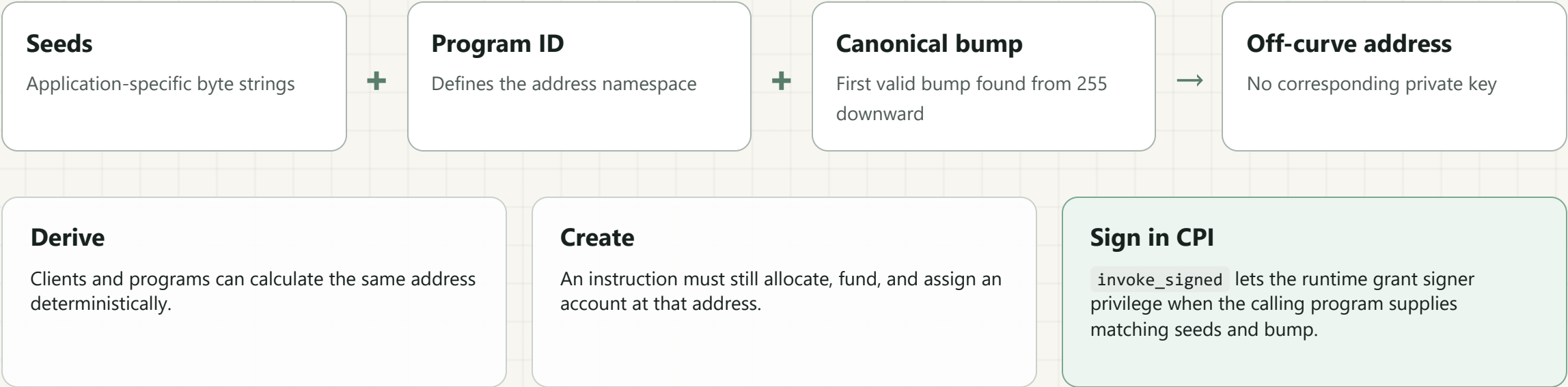
Concurrency condition

Non-conflicting transactions can execute in parallel because their account read and write sets are known in advance.

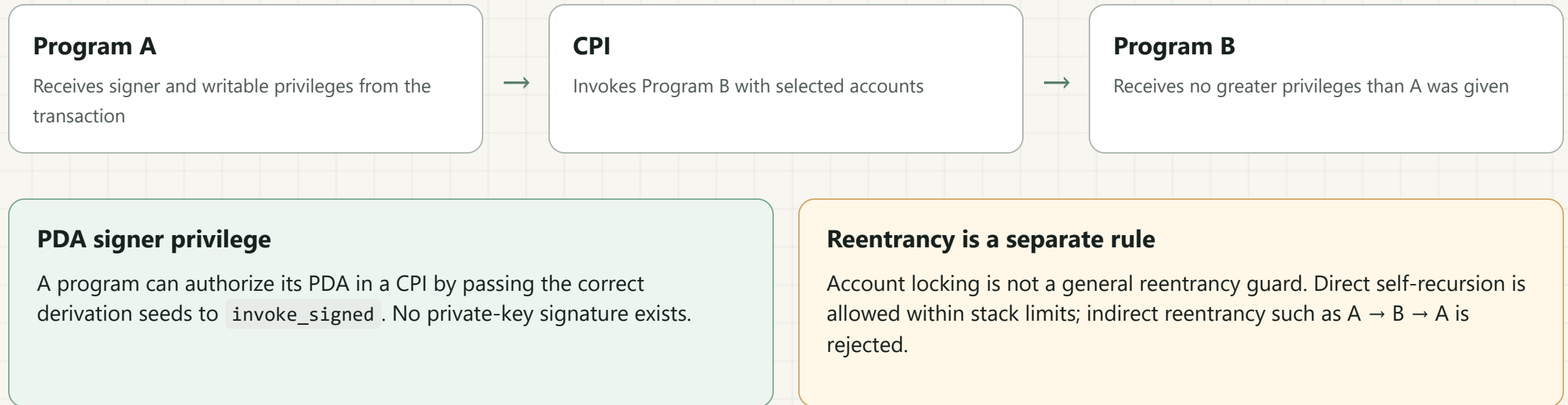
Practical constraint

Marking unnecessary accounts writable adds contention and grants more privilege than the instruction needs.

A PDA is derived locally. Derivation does not create an account.



CPI composes programs without allowing privilege escalation



Compute limits bound work; fees price signatures and requested priority

5,000

lamports per signature

Base transaction fee in the documented model.

200,000

default CUs

Per non-builtin instruction when no custom limit is set.

1,400,000

maximum CUs

Transaction compute-unit limit.

```
priority fee in lamports = ceil(  
  compute unit limit × compute unit price in micro-lamports ÷ 1,000,000  
)
```

Raising the compute-unit limit alone does not raise priority. A transaction requests priority by setting a compute-unit price, and it pays based on the requested limit rather than actual consumption.

Use the environment that matches the risk of the task

Localnet

Private validator for fast, repeatable development. State and local test SOL are disposable.

Devnet

Public development cluster with faucets and Explorer visibility. Devnet SOL has no monetary value, and cluster state may reset.

Mainnet beta

Production network using assets with real value. Session 1 work does not require mainnet deployment.

```
solana config get
solana config set --url localhost
solana config set --url devnet
solana balance
```

The project connects Rust program logic to generated client types and tests

Source and configuration

- `programs/hello_solana/src/lib.rs` : program instructions and account validation
- `Anchor.toml` : provider, wallet, program IDs, and test command
- `tests/hello_solana.ts` : client-side integration test

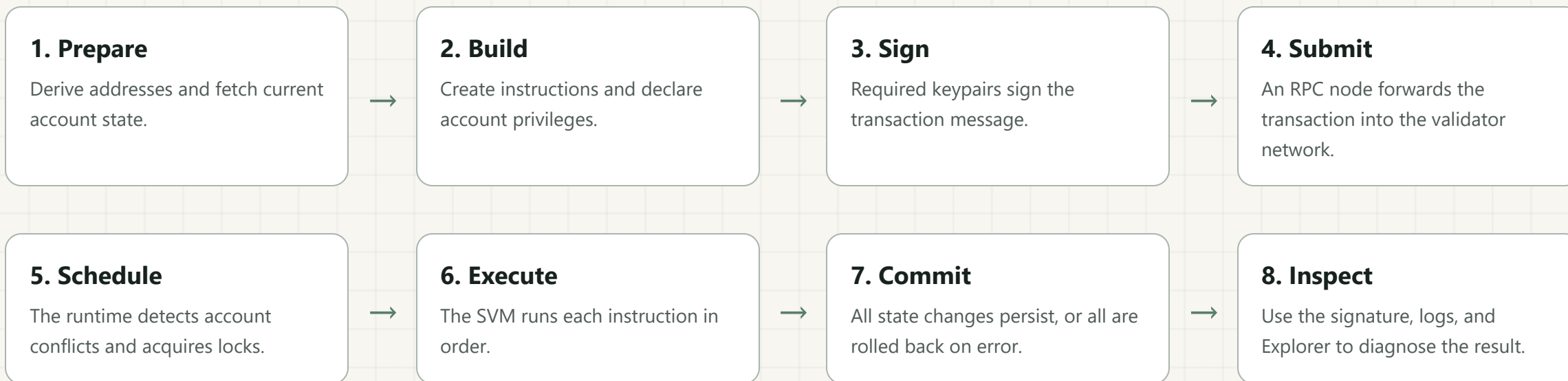
Generated output

- `target/idl/` : interface description
- `target/types/` : generated TypeScript types
- `target/deploy/` : deployable program and deployment keypair

```
anchor build
anchor keys list
anchor test
anchor logs
```

The deployment keypair establishes the program ID and is secret material. Never commit `*-keypair.json` , wallet keypairs, or generated deployment credentials.

From client intent to an atomic account update



A transaction signature identifies the submitted transaction. It is safe to publish when used as an intentional devnet example, but it is not a substitute for examining logs and account state.

Verify the active cluster, IDs, accounts, and logs before changing code

Diagnostic sequence

1. Check tool versions.
2. Run `solana config get`.
3. Confirm wallet and program IDs.
4. Reproduce locally with `anchor test`.
5. Read program logs and the exact transaction error.
6. Check compute use, funding, signer status, and writable accounts.

Official verification sources

- solana.com/docs/core/accounts
- solana.com/docs/core/transactions
- solana.com/docs/core/pda
- solana.com/docs/core/cpi
- solana.com/docs/core/fees
- anchor-lang.com/docs/references/cli